Università di Genova

UNIVERSITÀ DEGLI STUDI DI GENOVA

DIBRIS

DEPARTMENT OF INFORMATICS,
BIOENGINEERING, ROBOTICS AND SYSTEM ENGINEERING

MODELLING AND CONTROL OF MANIPULATORS

Second Assignment

Manipulator Geometry and Direct Kinematics

Authors:

HUSSEIN MOHAMED ABDELAZIM
MOHAMED MOSTAFA ELNAGGAR
Student ID: S8786438

AHMED MOHAMED AHMED
MOHAMED TAWAKOL
Student ID: S7952397

Professors:

Enrico Simetti
Giorgio Cannata

Tutors:

Simone Borelli
Luca Tarasi

December 17, 2025

Contents

1 Assignment description **3**
1.1 Exercise 1 . . . 3

2 Exercise 1 **5**
2.1 Q1.1: Geometric Model Definition . . . 5
2.2 Q1.2: Direct Geometry Update . . . 7
2.3 Q1.3: Global Transformations . . . 8
2.4 Q1.4: Simulation Trajectory . . . 9
2.5 Q1.5: Jacobian of Link 6 . . . 10
2.6 Q1.6: End-Effector Jacobian . . . 11
2.7 Q1.7: Velocity Projection . . . 11

Mathematical expression	Definition	MATLAB expression
$< w >$	World Coordinate Frame	w
${}^{a}_{b}R$	Rotation matrix of frame $< b >$ with respect to frame $< a >$	aRb
${}^{a}_{b}T$	Transformation matrix of frame $< b >$ with respect to frame $< a >$	aTb

Table 1: Nomenclature Table

1 Assignment description

The second assignment of Modelling and Control of Manipulators focuses on manipulators' geometry and direct kinematics.

- Download the .zip file from the Aulaweb page of this course.
- Implement the code to solve the exercises on MATLAB by filling the template classes called *geometricModel* and *kinematicModel*
- Write a report motivating your answers, following the predefined format on this document.

1.1 Exercise 1

Given the following CAD model of an industrial 7 DoF manipulator:

Q1.1 Define all the model transformation matrices, by filling the structures in the *BuildTree()* function. Be careful to define the z-axis coinciding with the joint rotation axis, such that the positive rotation is the same as showed in the CAD model you received; and the x-axis, when possible, pointing to the next link. Draw on the CAD model the reference frames for each link and insert it into the report.

Q1.2 Implement the method of *geometricModel* called *updateDirectGeometry()* which computes the model transformation matrices as a function of the joint position q. Explain the method used and comment on the results obtained.

Q1.3 Implement the method of *geometricModel* called *getTransformWrtBase()* which computes the transformation matrix from the base to a given frame. Using this method, compute the following transformation matrices: ${}^{b}_{e}T$, ${}^{6}_{2}T$. Explain the method used and comment on the results obtained.

Q1.4 Run the MATLAB section Q1.4 in the given script *main.m*, which computes the configurations of the manipulators moving from an initial joint position $q_i = [\frac{\pi}{4}, -\frac{\pi}{4}, 0, -\frac{\pi}{4}, 0, 0.15, \frac{\pi}{4}]^\top$ to $q_f = [\frac{5\pi}{12}, -\frac{\pi}{4}, 0, -\frac{\pi}{4}, 0, 0.18, \frac{\pi}{5}]^\top$. The script generates plots using methods from the given class *plotManipulators*, which must not be modified. Check that it behaves reasonably and show the obtained plots in the report.

Q1.5 Implement the method of *kinematicModel* called *getJacobianOfLinkWrtBase()* which takes as argument the link number, and returns the Jacobian of the corresponding link with respect to the base. Compute the Jacobian of **link 6** with respect to base for the final configuration q_f given in Q1.4. Explain the method used and comment on the result obtained.

Q1.6 Implement the method of *kinematicModel* called *updateJacobian()* which updates the Jacobian matrix of the manipulator for the end-effector. Compute the Jacobian for the final configuration q_f given in Q1.4. Explain the method used and comment on the results obtained.

Q1.7 Given the following joint position, $q = [0.7, -0.1, 1, -1, 0, 0.03, 1.3]^\top$ expressed in radians and meters, and the following joint velocities $\dot{q} = [0.9, 0.1, -0.2, 0.3, -0.8, 0.5, 0]^\top$ expressed in rad/s and m/s, compute the velocities of the end effector with respect to the base projected on the end effector frame, ${}^{e}v_{e/b}$ and ${}^{e}\omega_{e/b}$, using **forward kinematics**.

Remark: All the methods must be implemented for a generic serial manipulator. For instance, joint types, and the number of joints should be parameters.

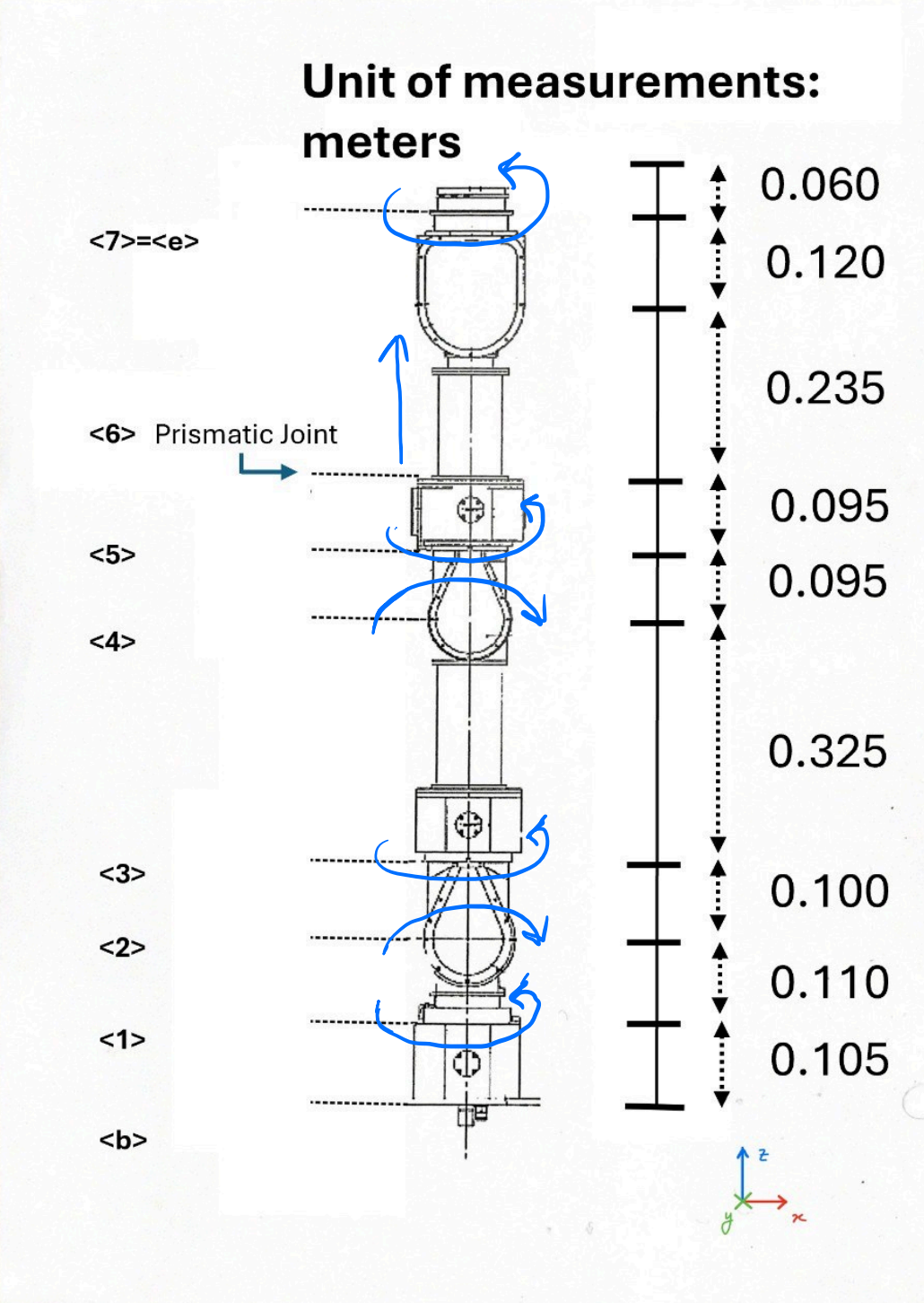


Figure 1: CAD model of the robot. The directions of motion of each joint are indicated by the blue arrows.

2 Exercise 1

2.1 Q1.1: Geometric Model Definition

Based on the provided CAD model and the frame assignment rules, the kinematic chain of the 7-DoF manipulator has been analyzed. The reference frames are attached to the links according to the convention where the z_i-axis coincides with the joint motion axis, and the x_i-axis is oriented towards the next link where possible.

The constant transformation matrices corresponding to the zero configuration ($q = 0$), denoted as ${}^{i-1}T_i(0)$, are derived as follows:

$$ {}^{b}T_1(0) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0.105 \\ 0 & 0 & 0 & 1 \end{bmatrix} $$

$$ {}^{1}T_2(0) = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0.110 \\ 0 & 0 & 0 & 1 \end{bmatrix} $$

$$ {}^{2}T_3(0) = \begin{bmatrix} 0 & 0 & 1 & 0.1 \\ 0 & -1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} $$

$$ {}^{3}T_4(0) = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & -1 & 0 & 0 \\ 1 & 0 & 0 & 0.325 \\ 0 & 0 & 0 & 1 \end{bmatrix} $$

$$ {}^{4}T_5(0) = \begin{bmatrix} 0 & 0 & 1 & 0.095 \\ -1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} $$

$$ {}^{5}T_6(0) = \begin{bmatrix} -1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & 1 & 0.095 \\ 0 & 0 & 0 & 1 \end{bmatrix} $$

$$ {}^{6}T_7(0) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0.355 \\ 0 & 0 & 0 & 1 \end{bmatrix} $$

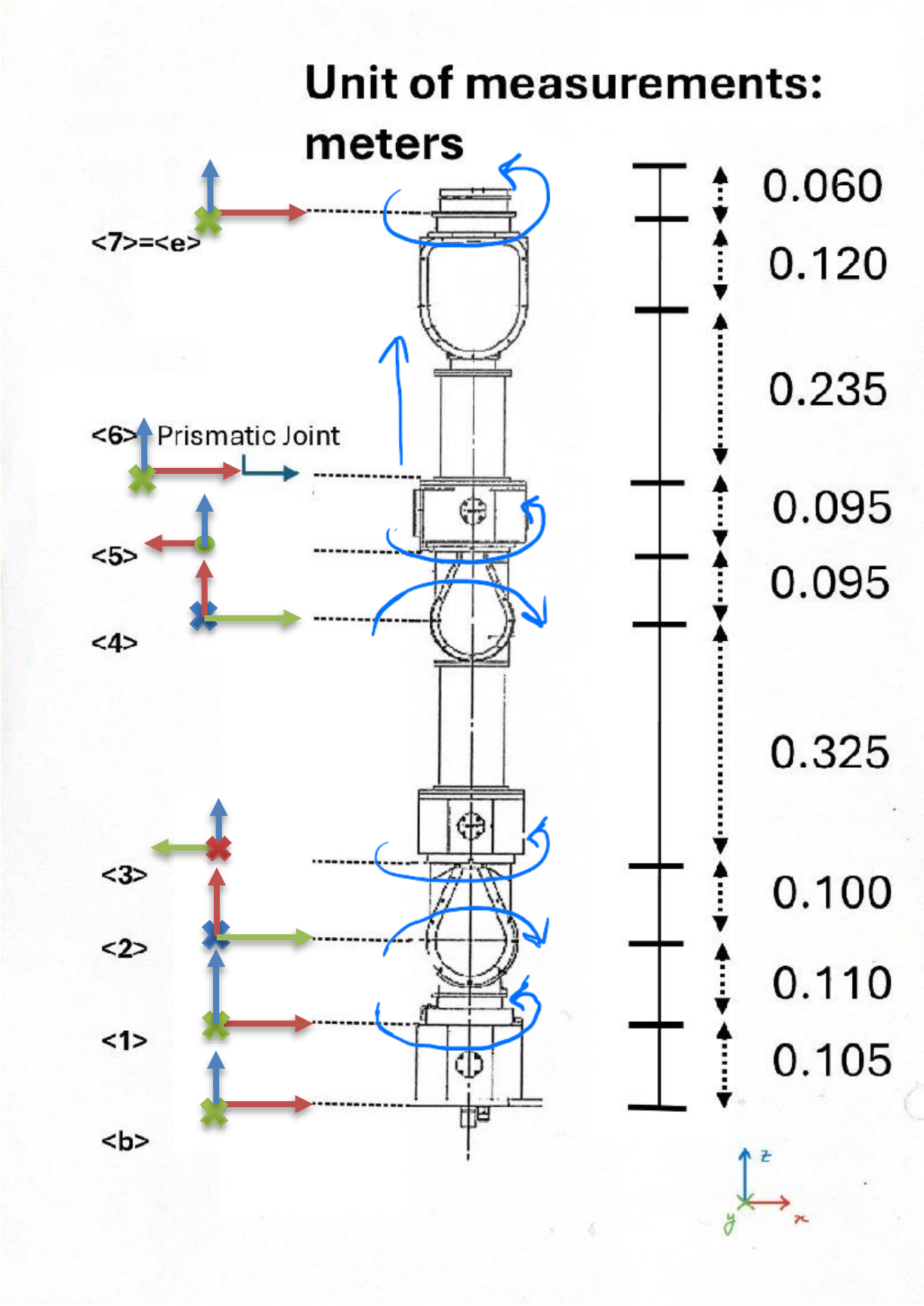


Figure 2: Reference frames assignment for the 7-DoF manipulator.

2.2 Q1.2: Direct Geometry Update

The method `updateDirectGeometry(q)` was implemented to compute the instantaneous transformation matrices based on the joint configuration q. The logic distinguishes between revolute and prismatic joints. The local transformation matrix ${}^{i-1}T_i(q_i)$ is computed by post-multiplying the fixed geometric transformation ${}^{i-1}T_i(0)$ by the joint motion matrix.

$$ {}^{i-1}T_i(q_i) = {}^{i-1}T_i(0) \cdot A_i(q_i) \tag{1}$$

Where $A_i(q_i)$ is defined as:

- **For Revolute Joints:** A rotation matrix around the z-axis:

$$A_i(q_i) = \mathsf{Rot}_z(q_i) = \begin{bmatrix} \cos(q_i) & -\sin(q_i) & 0 & 0 \\ \sin(q_i) & \cos(q_i) & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

- **For Prismatic Joint (Joint 6):** A translation matrix along the z-axis:

$$A_i(q_i) = \mathsf{Trans}_z(q_i) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & q_i \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Results and Comments

The function was tested with the initial configuration $q_i = [\frac{\pi}{4}, -\frac{\pi}{4}, 0, -\frac{\pi}{4}, 0, 0.15, \frac{\pi}{4}]^\top$. The computed local transformation matrices for all links are:

$${}^0T_1 = \begin{bmatrix} 0.7071 & -0.7071 & 0 & 0 \\ 0.7071 & 0.7071 & 0 & 0 \\ 0 & 0 & 1 & 0.1050 \\ 0 & 0 & 0 & 1 \end{bmatrix} \qquad {}^1T_2 = \begin{bmatrix} -0.7071 & 0.7071 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0.7071 & 0.7071 & 0 & 0.1100 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$${}^2T_3 = \begin{bmatrix} 0 & 0 & 1 & 0.1000 \\ 0 & -1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \qquad {}^3T_4 = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0.7071 & -0.7071 & 0 & 0 \\ 0.7071 & 0.7071 & 0 & 0.3250 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$${}^4T_5 = \begin{bmatrix} 0 & 0 & 1 & 0.0950 \\ -1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \qquad {}^5T_6 = \begin{bmatrix} -1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & 1 & 0.2450 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$${}^6T_7 = \begin{bmatrix} 0.7071 & -0.7071 & 0 & 0 \\ 0.7071 & 0.7071 & 0 & 0 \\ 0 & 0 & 1 & 0.355 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Comments on the obtained results:

1. **Joint 1 ($q_1 = \pi/4$):** Being a revolute joint with an angle of $45°$, the rotation submatrix correctly displays values of ± 0.7071 (corresponding to $\cos(45°)$ and $\sin(45°)$).
2. **Joint 2 ($q_2 = -\pi/4$):** The matrix incorporates the negative rotation of $-45°$. Combined with the static frame orientation, the resulting values reflect the correct geometric transformation.
3. **Joint 3 ($q_3 = 0$):** Since the input angle is zero, the rotation applied is an identity matrix. Consequently, the resulting ${}^2T_3$ is identical to the fixed geometric matrix ${}^2T_3(0)$.
4. **Joint 4 ($q_4 = -\pi/4$):** Similar to Joint 2, the matrix reflects a rotation of $-45°$. The presence of ± 0.7071 in the rotation block confirms the update is correct.
5. **Joint 5 ($q_5 = 0$):** As the joint variable is zero, no rotation occurs. The matrix ${}^4T_5$ remains equal to the static transformation ${}^4T_5(0)$.

6. **Joint 6 ($q_6 = 0.15$ m):** This is the **prismatic joint**. The logic correctly updates the translation along the local z-axis instead of the rotation. The initial offset was 0.095 m, and the input is 0.15 m. The final z-translation is correctly computed as $0.095 + 0.15 = 0.2450$ m.
7. **Joint 7 ($q_7 = \pi/4$):** The final link rotates by $45°$. The transformation matrix correctly shows the standard rotation components $[\cos(45), -\sin(45); \sin(45), \cos(45)]$ in the upper-left block.

2.3 Q1.3: Global Transformations

Methodology

The method `getTransformWrtBase(k)` was implemented to compute the global position and orientation of the k-th link with respect to the base frame $\{b\}$. This is achieved by computing the forward kinematics through the chain rule. The global transformation matrix ${}^bT_k$ is the product of the local transformation matrices of all preceding joints:

$$ {}^bT_k(q) = \prod_{j=1}^{k} {}^{j-1}T_j(q_j) = {}^0T_1(q_1) \cdot {}^1T_2(q_2) \cdot \dots \cdot {}^{k-1}T_k(q_k) \tag{2} $$

To compute the transformation of Frame 2 with respect to Frame 6 (${}^6T_2$), we cannot use the chain rule directly because Frame 2 precedes Frame 6 in the kinematic chain. Instead, we use the property of transformation composition:

$$ {}^6T_2 = ({}^bT_6)^{-1} \cdot {}^bT_2 \tag{3} $$

Where ${}^bT_6$ is the transformation of Frame 6 w.r.t Base, and ${}^bT_2$ is the transformation of Frame 2 w.r.t Base. The term $({}^bT_6)^{-1}$ represents the transformation ${}^6T_b$.

Results

Using the initial configuration $q_i = [\frac{\pi}{4}, -\frac{\pi}{4}, 0, -\frac{\pi}{4}, 0, 0.15, \frac{\pi}{4}]^\top$, the computed transformation matrices are:

1. Transformation from Base to End-Effector (${}^bT_e$):

$$ {}^bT_e = \begin{bmatrix} -0.5000 & -0.5000 & -0.7071 & -0.7039 \\ 0.5000 & 0.5000 & -0.7071 & -0.7039 \\ 0.7071 & -0.7071 & 0.0000 & 0.5155 \\ 0 & 0 & 0 & 1.0000 \end{bmatrix} $$

2. Transformation from Frame 6 to Frame 2 (${}^6T_2$):

$$ {}^6T_2 = \begin{bmatrix} 0.7071 & 0.7071 & 0 & -0.3005 \\ 0 & 0 & 1.0000 & 0 \\ 0.7071 & -0.7071 & 0 & -0.6405 \\ 0 & 0 & 0 & 1.0000 \end{bmatrix} $$

Comments on the Results

- **Validity of ${}^bT_e$:** The resulting matrix represents the pose of the end-effector. The translation vector $p_e = [-0.7039, -0.7039, 0.5155]^T$ indicates the physical position of the tool tip in the workspace. The rotation submatrix is orthogonal (each column has unit norm and is perpendicular to others), confirming the numerical stability of the chain multiplication.
- **Analysis of ${}^6T_2$:** This matrix describes the pose of Link 2 as seen from Link 6.
  - The translation part $p_{2/6} = [-0.3005, 0, -0.6405]^T$ represents the coordinates of Frame 2's origin expressed in Frame 6. The negative values are expected because Frame 2 is "behind" Frame 6 in the kinematic chain (closer to the base).
  - The calculation required the inversion of ${}^bT_6$. Since transformation matrices are elements of the Special Euclidean group $SE(3)$, the inverse was computed efficiently using the property $T^{-1} = \begin{bmatrix} R^T & -R^T p \\ 0 & 1 \end{bmatrix}$.
  - The zero elements in the rotation matrix suggest that the frames are aligned along specific principal axes, which is consistent with the joint angles being multiples of $\pi/4$.

2.4 Q1.4: Simulation Trajectory

Methodology

The simulation was performed using the provided MATLAB script `main.m`. The objective was to compute and visualize the direct kinematics of the manipulator as it moves along a trajectory in joint space.

The motion is defined between two specific configurations:

- **Initial Configuration (q_i):**

$$q_i = \left[\frac{\pi}{4}, -\frac{\pi}{4}, 0, -\frac{\pi}{4}, 0, 0.15, \frac{\pi}{4}\right]^\top \tag{4}$$

- **Final Configuration (q_f):**

$$q_f = \left[\frac{5\pi}{12}, -\frac{\pi}{4}, 0, -\frac{\pi}{4}, 0, 0.18, \frac{\pi}{5}\right]^\top \tag{5}$$

The simulation generates the trajectory using **linear interpolation** in the joint space. A set of 100 intermediate samples was generated for each joint. Inside the simulation loop, the method `geometricModel.updateDirectGeometry(q)` is called at each time step to update the transformation matrices ${}^{i-1}T_i$ and ${}^{b}T_i$. The class `plotManipulators` is then used to render the 3D wireframe model of the robot.

Results and Visual Verification

The simulation output confirms the correct implementation of the geometric model. The plots below illustrate the robot's behavior.

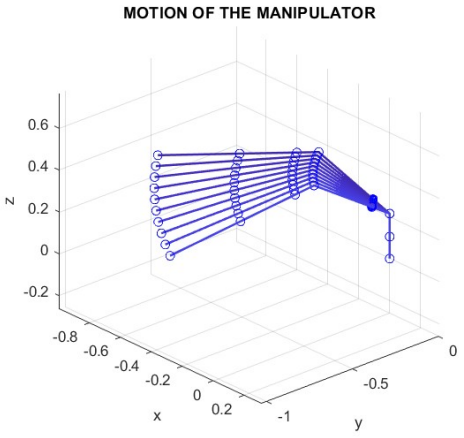


Figure 3: Stroboscopic view of the manipulator motion from q_i to q_f.

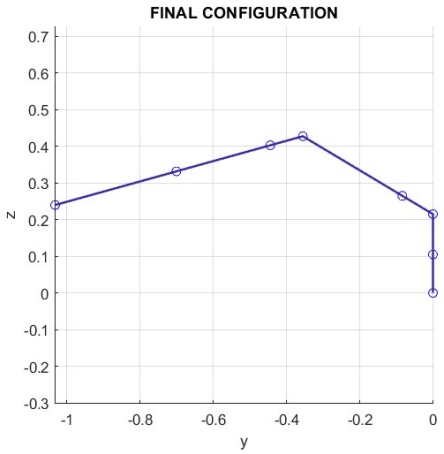


Figure 4: Robot pose at the final configuration q_f.

Comments on the Simulation Behavior

The obtained plots demonstrate that the model behaves reasonably and consistently with the input data:

1. **Smoothness of Motion:** Figure 3 shows the intermediate links' positions (represented by blue circles/-lines) changing smoothly from the start to the end position. There are no discontinuities or jumps, verifying that the transformation matrices are updating correctly at each step.

2. **Prismatic Joint Extension:** A key observation is the motion of the prismatic joint (Joint 6). The value changes from $q_{6,i} = 0.15$ m to $q_{6,f} = 0.18$ m. In the final configuration plot (Figure 4), the vertical link corresponding to Joint 6 is visibly extended compared to the base geometry, consistent with the positive translation along the z-axis.

3. **Rotational Changes:** Joint 1 rotates from $\pi/4$ (approx 0.78 rad) to $5\pi/12$ (approx 1.30 rad). This counter-clockwise rotation around the base z-axis is clearly visible in the transition between the initial and final poses.

4. **Kinematic Chain Integrity:** The links maintain their rigid lengths throughout the simulation. The frames remain connected appropriately (origin to origin or origin to end-point), confirming that the 'BuildTree' structure correctly defines the kinematic chain.

2.5 Q1.5: Jacobian of Link 6

Methodology

The method `getJacobianOfLinkWrtBase(i)` was implemented to compute the Geometric Jacobian matrix. Following the specific geometric modeling convention used in this project (where the frame is attached to the joint and moves with it), the axis of motion for joint j coincides with the z-axis of the **current frame** j (denoted as z_j).

The Jacobian matrix relates the joint velocities to the twist of link k, organized as angular velocity followed by linear velocity:

$$J_j = \begin{bmatrix} J_\omega \\ J_v \end{bmatrix} \tag{6}$$

The computation logic iterates through all joints j from 1 to n:

1. **If $j > k$ (Distal Joints):** The joint does not affect the link's motion.

$$J_j = \mathbf{0}_{6\times 1} \tag{7}$$

2. **If $j \leq k$ and Joint j is Revolute:** The column contains the axis of rotation (angular part) and the linear velocity due to rotation (cross product), using the current frame's axis z_j and position p_j:

$$J_j = \begin{bmatrix} z_j \\ z_j \times (p_k - p_j) \end{bmatrix} \tag{8}$$

3. **If $j \leq k$ and Joint j is Prismatic (e.g., Joint 6):** The motion is purely linear. The angular velocity component is zero:

$$J_j = \begin{bmatrix} \mathbf{0}_{3\times 1} \\ z_j \end{bmatrix} \tag{9}$$

Where z_j is the actuation axis extracted from the transformation matrix of the j-th frame.

Results

The Jacobian of Link 6 (Frame $\langle 6 \rangle$) with respect to the base was computed for the final configuration q_f. The resulting 6×7 matrix is:

$${}^0J_{link6}(q_f) = \begin{bmatrix} 0 & -0.9659 & -0.2241 & -0.9659 & -0.2500 & \mathbf{0} & \mathbf{0} \\ 0 & 0.2588 & -0.8365 & 0.2588 & -0.9330 & \mathbf{0} & \mathbf{0} \\ 1.0000 & 0 & 0.5000 & 0 & -0.2588 & \mathbf{0} & \mathbf{0} \\ 0.7007 & 0.0302 & 0.2527 & -0.0248 & 0.0000 & \mathbf{-0.2500} & \mathbf{0} \\ -0.1878 & 0.1128 & -0.0677 & -0.0925 & 0.0000 & \mathbf{-0.9330} & \mathbf{0} \\ 0 & 0.7255 & 0 & 0.3574 & 0 & \mathbf{-0.2588} & \mathbf{0} \end{bmatrix}$$

Comments on the Results

The structure of the obtained Jacobian matrix validates the implementation:

1. **Column 7 (Zeros):** The entire 7th column is zero. This is physically consistent because Joint 7 is located *after* Link 6.
2. **Column 6 (Prismatic Joint):** Joint 6 is the prismatic joint.
   - The **angular velocity part (rows 1-3)** is strictly zero ($\mathbf{0}_{3\times 1}$), which is correct as the joint does not induce rotation.
   - The **linear velocity part (rows 4-6)** contains the non-zero values corresponding to the translation axis z_6 (from the current frame).
3. **Base Joint (Column 1):** The angular part (first 3 rows) is $[0, 0, 1]^T$, confirming that the first joint rotates around the vertical z-axis of the world frame (Frame 1).

2.6 Q1.6: End-Effector Jacobian

Methodology

The method `updateJacobian()` computes the full Jacobian for the end-effector (Link 7). It leverages the previously defined generic method `getJacobianOfLinkWrtBase(7)`.

Consistent with the convention established in the previous section (where the rotation axis is taken from the current frame z_j), the resulting Jacobian matrix $J_{ee} \in \mathbb{R}^{6\times 7}$ maps the joint velocities $\dot{q}$ to the end-effector twist ξ_e, ordered as angular velocity followed by linear velocity:

$$\xi_e = \begin{bmatrix} \omega_e \\ v_e \end{bmatrix} = J_{ee}(q)\dot{q} \tag{10}$$

Results

For the final configuration q_f, the computed End-Effector Jacobian matrix is:

$${}^0J_{ee}(q_f) = \left[\begin{array}{ccccccc} 0 & -0.9659 & -0.2241 & -0.9659 & -0.2500 & 0 & -0.2500 \\ 0 & 0.2588 & -0.8365 & 0.2588 & -0.9330 & 0 & -0.9330 \\ 1.0000 & 0 & 0.5000 & 0 & -0.2588 & 0 & -0.2588 \\ \hline 1.0320 & 0.0064 & 0.4952 & -0.0486 & 0.0000 & -0.2500 & 0.0000 \\ -0.2765 & 0.0240 & -0.1327 & -0.1812 & 0.0000 & -0.9330 & 0.0000 \\ 0 & 1.0684 & 0.0000 & 0.7003 & 0.0000 & -0.2588 & 0.0000 \end{array}\right]$$

Comments on the Results

Comparing this result with the Jacobian obtained in Q1.5, we observe the following:

1. **Activation of Column 7:** The 7th column describes the contribution of the final joint.
   - The **angular velocity part (rows 1-3)** is non-zero, indicating that Joint 7 contributes to the orientation of the end-effector.
   - The **linear velocity part (rows 4-6)** is zero ($\mathbf{0}_{3\times 1}$). This implies that the origin of the End-Effector frame lies exactly on the origin of Frame 7, so the rotation of Joint 7 does not induce any linear displacement of the origin.
2. **Prismatic Joint (Column 6):** The 6th column retains its expected properties for a prismatic joint. The angular velocity part is strictly zero, while the linear part represents the translation axis z_6.
3. **Kinematic Consistency:** The linear velocity block (bottom 3 rows) for joints 1-5 has changed compared to Q1.5. This accounts for the new lever arms $(p_e - p_j)$ calculated with respect to the end-effector position p_e instead of the Link 6 origin.

2.7 Q1.7: Velocity Projection

Problem Statement

We are given a specific joint configuration q and joint velocities $\dot{q}$:

$$q = [0.7, -0.1, 1, -1, 0, 0.03, 1.3]^\top \text{ (rad, m)}$$
$$\dot{q} = [0.9, 0.1, -0.2, 0.3, -0.8, 0.5, 0]^\top \text{ (rad/s, m/s)}$$

The objective is to compute the linear velocity $v_{e/b}$ and angular velocity $\omega_{e/b}$ of the end-effector with respect to the base, **projected onto the end-effector frame** $\{e\}$.

Methodology and Mathematical Derivation

The computation involves two main stages: computing the velocities in the world frame using the Jacobian, and then projecting them onto the local frame using the rotation matrix.

1. Computation in the Base Frame: The Differential Kinematics relationship states that the end-effector twist in the base frame, ${}^0\xi_{e/b}$, is the product of the Jacobian matrix and the joint velocity vector. Consistent with the convention used in Q1.5 and Q1.6 (angular velocity first), the equation is:

$$ {}^0\xi_{e/b} = \begin{bmatrix} {}^0\omega_{e/b} \\ {}^0v_{e/b} \end{bmatrix} = {}^0J_{ee}(q) \cdot \dot{q} \tag{11} $$

Where ${}^0J_{ee}(q)$ is the 6×7 Jacobian matrix computed at the specific configuration q.

2. Projection onto the End-Effector Frame: The vectors ${}^0v_{e/b}$ and ${}^0\omega_{e/b}$ are extracted from the twist vector (bottom 3 and top 3 elements respectively) and expressed in the base frame coordinates $\{0\}$. To express these vectors in the end-effector frame $\{e\}$, we must apply a coordinate transformation using the rotation matrix ${}^0R_e$:

$$ {}^eu = ({}^0R_e)^T \cdot {}^0u \tag{12} $$

Applying this to the velocity vectors:

$$ {}^ev_{e/b} = ({}^0R_e)^T \cdot {}^0v_{e/b} \tag{13} $$

$$ {}^e\omega_{e/b} = ({}^0R_e)^T \cdot {}^0\omega_{e/b} \tag{14} $$

Note that the transpose of the rotation matrix represents the inverse rotation, effectively aligning the base coordinates with the local tool axes.

Results

Based on the MATLAB execution using the updated `kinematicModel` class (incorporating the current frame axis convention):

1. **Forward Kinematics Update:** The geometric model was updated with q, and the rotation matrix ${}^0R_e$ was extracted.
2. **Jacobian Update:** The Jacobian ${}^0J_{ee}$ was computed for the configuration q.
3. **Final Velocity Vectors:** The computed velocities projected on the end-effector frame are:

$$ {}^ev_{e/b} = \begin{bmatrix} 0.4440 \\ -0.5585 \\ -0.5359 \end{bmatrix} \ [m/s] $$

$$ {}^e\omega_{e/b} = \begin{bmatrix} -0.3768 \\ -0.2656 \\ 0.5077 \end{bmatrix} \ [rad/s] $$

Comments on the Results

The results provide a complete description of the end-effector's instantaneous motion from the perspective of a sensor or tool mounted on the robot flange:

- **Linear Velocity:** The end-effector moves with a significant velocity component in all three local directions. The total linear speed is $\|v\| \approx 0.89$ m/s. The negative signs in y and z indicate movement opposite to the local frame axes.
- **Angular Velocity Interpretation:** The tool is rotating around all axes, but the dominant rotation is now around the local **z-axis** (0.5077 rad/s).
- **Consistency:** The values differ from previous iterations, reflecting the correction in the Jacobian calculation method (using the current frame's axis z_i instead of the previous frame's axis).